

**Research Institute
Cyber Defence**
University of the Bundeswehr Munich

Hardware Specification

SHARMONY

A unified, crypto-agile hash accelerator

SHA-2 · SHA-3 · SHAKE · cSHAKE in a shared 64-bit datapath,
duet SHA-224/256 execution, mid-state caching,
verified against NIST CAVP test vectors.

Classification: Public

This document describes the SHARMONY hardware. It is released together with the RTL and its verification environment under the **Apache License, Version 2.0**, and may be freely used, reproduced, and distributed under the terms of that license.

Document Information

Document ID: FORTRESS-UniBwM-001

Version: v1.0

Status: Released

Release Date: 22.07.2026

Prepared by

Liga Anwar

Carlos Andres Lara-Nino

Jong-Yeon Park

Michael Hutter



Document Control

Field	Value
Document Title	Hardware Specification
Product / Module	SHARMONY
Document ID	FORTRESS-UniBwM-001
Version	v1.0
Status	Released
Release Date	22.07.2026
Authors	Liga Anwar, Carlos Andres Lara-Nino, Jong-Yeon Park, and Michael Hutter
Owning Organization	ESSEC Lab, Research Institute CODE, University of the Bundeswehr Munich
Classification	Public
License	Apache License, Version 2.0
Repository	https://github.com/essec-lab/sharmony

Revision History

Version	Date	Author	Description of Change	Status
0.1	11.06.2026	Liga Anwar	Initial release.	Draft
0.2	20.06.2026	Liga Anwar	Improved version, added waveforms.	Draft
0.3	05.07.2026	Liga Anwar	Consistency and editorial fixes; draft prepared for review.	Draft
1.0	22.07.2026	Liga Anwar	Apache-2.0 licensing; RTL-verified interface description; updated figures and numbers; editorial polish for public release.	Released

Contents

1	Scope	9
1.1	In Scope	9
1.2	Out of Scope	9
2	System Overview	9
2.1	Key Characteristics	9
2.2	Supported Modes	10
3	Hardware Architecture	11
3.1	Block Diagram	11
3.2	Clocking and Reset	11
3.3	Configuration Parameters	12
3.4	Requirements	13
4	Interfaces and Connectors	13
4.1	Interface List	13
4.2	Top-Level Signal List	14
4.3	Operation Sequence	15
4.4	Interface Timing Diagrams	16
4.4.1	Reading the Diagrams: the Registered I/O Boundary	16
4.4.2	SHA-512 Hash Transaction	16
4.4.3	SHA3-256 Hash Transaction	17
4.4.4	Message-Input Handshake	18
4.4.5	Output Back-Pressure	18
4.4.6	Duet Execution	19
4.4.7	XOF Squeeze	20
4.4.8	Invalid Access	21
4.4.9	Zeroization	21
4.5	State Management	22
4.5.1	Context Switching (state_save / state_load)	22
4.5.2	Mid-State Caching (state_cache)	23
4.6	State-Management Timing	23
5	Implementation Results	26
5.1	FPGA Resource Utilization	26
5.2	Timing and I/O Delay	27
5.3	ASIC Area (Gate Equivalents)	27
5.4	Latency	28

5.5	Throughput	28
6	Standards Compliance	29
7	Test and Verification Requirements	29
7.1	Verification Methodology	29
7.2	Available Tests	29
7.3	Verification Matrix	30
8	References	31
A	Appendices	32
A.1	Schematic References	32
A.2	Repository Structure	34
A.3	Running the Test Suite	34

List of Figures

1	Top-level block diagram of SHARMONY: shared padding module, shared FSM control, and the shared SHA-2/SHA-3 datapath.	11
2	SHA-512 hash of the empty message.	17
3	SHA3-256 hash of the empty message.	18
4	Message-input valid/ready handshake with stalls.	18
5	Digest-output handshake under back-pressure.	19
6	Duet execution: two independent SHA-256 streams on the split bus.	20
7	SHAKE256 extendable-output (XOF) squeeze.	20
8	Invalid-access handling: refused pre-start beat and ignored restarts.	21
9	Zeroization: five-cycle scrub of the register chains.	22
10	Mid-state save (SHA-512).	24
11	Mid-state resume (SHA-512).	24
12	Cache-save via state_cache (SHA-512).	25
13	Cache-resume from the on-chip mid-state (SHA-512).	26
14	Datapath schematic: message-scheduler half of the register bank.	32
15	Datapath schematic: compression half and digest bank.	33
16	Constants ROM memory map.	33

List of Tables

1	Supported hash and XOF modes with their mode_e encodings.	10
2	Hardware requirements and their verification approach.	13
3	Top-level interface groups.	14
4	Top-level signal list of essec_sharmony_top.	14
5	State-transfer operations selected by state_load and state_save.	22
6	Post-synthesis FPGA resource utilization (Artix-7, xc7a200t fbg484-3).	26
7	Timing and I/O boundary characteristics on the Artix-7 target.	27
8	ASIC area estimate in gate equivalents (Nangate 45 nm, typical corner).	27
9	Single-block latency (SHA-224, one 512-bit block).	28
10	Throughput per mode.	28
11	Applicable standards and conformance status.	29
12	Available tests in the verification suite.	29
13	Verification matrix: requirement coverage by test.	30

Definitions and Abbreviations

Term	Definition
ASIC	Application-Specific Integrated Circuit
BRAM	Block Random Access Memory
CAVP	Cryptographic Algorithm Validation Program
CCA	Chosen-Ciphertext Attack
CPA	Correlation Power Analysis
DMA	Direct Memory Access
DPA	Differential Power Analysis
DRBG	Deterministic Random Bit Generator
DSA	Digital Signature Algorithm
DSP	Digital Signal Processing
EMA	Electromagnetic Analysis
EMFI	Electromagnetic Fault Injection
FF	Flip-Flop
FIA	Fault Injection Attack
FIPS	Federal Information Processing Standards
FORS	Forest of Random Subsets
FORTRESS	Fully Optimized Root of Trust for Robust Embedded Secure Systems
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GE	Gate Equivalent (area of a 2-input NAND)
HBS	Hash-Based Signature
HDL	Hardware Description Language
HLS	High-Level Synthesis
HMAC	Hash-Based Message Authentication Code
HSS	Hierarchical Signature System
HW/SW	Hardware/Software
I/O	Input/Output
IP	Intellectual Property
ISA	Instruction Set Architecture
IV	Initial Value / Initialization Vector
KAT	Known Answer Test
KEM	Key Encapsulation Mechanism
KMAC	KECCAK Message Authentication Code
LM-OTS	Leighton-Micali One-Time Signature
LMS	Leighton-Micali Signature
LUT	Look-Up Table
MAC	Message Authentication Code
MCU	Microcontroller Unit
NIST	National Institute of Standards and Technology
OTP	One-Time Programmable
OTS	One-Time Signature
PKE	Public-Key Encryption
PPA	Power, Performance, and Area
PQC	Post-Quantum Cryptography
PRF	Pseudorandom Function
PRNG	Pseudorandom Number Generator

Term	Definition
PUF	Physical Unclonable Function
RAM	Random Access Memory
RISC-V	Reduced Instruction Set Computer Five
ROM	Read-Only Memory
RoT	Root of Trust
RTL	Register Transfer Level
SCA	Side-Channel Attack
SHA	Secure Hash Algorithm
SHAKE	Secure Hash Algorithm KECCAK
cSHAKE	Customizable SHAKE (NIST SP 800-185)
SLH-DSA	Stateless Hash-Based Digital Signature Algorithm
SoC	System-on-Chip
SPA	Simple Power Analysis
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
TRNG	True Random Number Generator
TVLA	Test Vector Leakage Assessment
UART	Universal Asynchronous Receiver-Transmitter
VHDL	VHSIC Hardware Description Language
WOTS+	Winternitz One-Time Signature Plus
XMSS	eXtended Merkle Signature Scheme
XOF	Extendable-Output Function

1 Scope

This document specifies the hardware design, interfaces, and verification requirements of SHARMONY, a unified, crypto-agile hash accelerator for the SHA-2, SHA-3, SHAKE, and cSHAKE families. It is intended for hardware designers integrating SHARMONY into an FPGA or ASIC system, verification engineers running or extending the accompanying test suite, and researchers reproducing the reported implementation results. It applies to version 1.0 of the SHARMONY specification and is maintained under the document-control policies of Research Institute CODE. Errors and suggestions are welcome via the repository issue tracker (<https://github.com/essec-lab/sharmony>).

1.1 In Scope

- Functional specification of the supported hash and XOF modes.
- Top-level RTL interface: streaming message input, streaming digest/XOF output, and control signals.
- State management features: mid-state save/load (context switching) and on-chip mid-state caching.
- Compile-time configuration options and their resource, timing, and latency trade-offs.
- Verification methodology and test requirements (NIST CAVP).

1.2 Out of Scope

- Software driver and firmware specification.
- Side-channel and fault-injection countermeasure evaluation (SCA/FIA hardening is a research topic outside this revision).
- Board-level integration, packaging, and physical hardware items (the deliverable is an RTL IP core).

2 System Overview

SHARMONY is a unified, crypto-agile hash accelerator that implements the SHA-2, SHA-3, SHAKE, and cSHAKE families on a single shared 64-bit datapath. A common padding front-end serves every algorithm, and a single 25×64 -bit register bank, holding either the SHA-2 message schedule and working variables or the 1,600-bit `KECCAK` sponge state, is time-multiplexed between the two hash families rather than duplicated; one mode is active at a time. In the SHA-224/SHA-256 modes the datapath additionally splits into two 32-bit lanes, so two independent message streams can be processed in parallel (*duet* execution). This section summarizes the core characteristics and the supported mode set; the remaining sections specify the interface, behavior, and verification requirements in detail.

2.1 Key Characteristics

- Single RTL core covering the SHA-2, SHA-3, SHAKE, and cSHAKE families on a shared 64-bit streaming datapath.
- Concurrent two-lane SHA-224/SHA-256 duet execution: in the 32-bit modes the 64-bit datapath splits into two 32-bit lanes, hashing two independent message streams in the same number of cycles as a single hash and doubling the effective throughput.
- Customizable XOF operation (cSHAKE128/cSHAKE256): domain separation via the function-name (N) and customization (S) strings, whose SP 800-185 prefix encoding (bytepad) is performed by the host and absorbed as ordinary message input.
- Resumable hashing via exportable and re-importable mid-state (`state_save/state_load`): the SHA-2 eight-word chaining value or the full 25-lane `KECCAK` state.

- On-chip single-slot mid-state cache (state_cache) that retains a computed prefix state for reuse without any bus transfer, suited to fixed-prefix PQC constructions such as SLH-DSA and XMSS.
- Ready/valid back-pressure on both the message-input and the digest/XOF-output streams.
- Bounded zeroization: all sensitive registers are scrubbed within a fixed five-cycle window.
- Single clock domain; fully synchronous, active-low reset (resetsn, Section 3.2).
- Conformance verified against NIST CAVP test vectors.

2.2 Supported Modes

The top-level mode input uses the mode_e enumeration from essec_sharmony_pkg.sv.

Table 1: Supported hash and XOF modes with their mode_e encodings.

Family	Variant	RTL identifier	Encoding	Digest size	Block / Rate	Capacity
SHA-2	SHA-224	HASH_SHA2_224	4'h0	224 bit	512 bit	—
SHA-2	SHA-256	HASH_SHA2_256	4'h1	256 bit	512 bit	—
SHA-2	SHA-384	HASH_SHA2_384	4'h2	384 bit	1024 bit	—
SHA-2	SHA-512	HASH_SHA2_512	4'h3	512 bit	1024 bit	—
SHA-2	SHA-512/224	HASH_SHA2_512_224	4'h4	224 bit	1024 bit	—
SHA-2	SHA-512/256	HASH_SHA2_512_256	4'h5	256 bit	1024 bit	—
SHA-3	SHA3-224	HASH_SHA3_224	4'h6	224 bit	1152 bit	448 bit
SHA-3	SHA3-256	HASH_SHA3_256	4'h7	256 bit	1088 bit	512 bit
SHA-3	SHA3-384	HASH_SHA3_384	4'h8	384 bit	832 bit	768 bit
SHA-3	SHA3-512	HASH_SHA3_512	4'h9	512 bit	576 bit	1024 bit
SHAKE	SHAKE128	XOF_SHAKE128	4'hA	variable	1344 bit	256 bit
SHAKE	SHAKE256	XOF_SHAKE256	4'hB	variable	1088 bit	512 bit
cSHAKE	cSHAKE128	XOF_CSHAKE128	4'hC	variable	1344 bit	256 bit
cSHAKE	cSHAKE256	XOF_CSHAKE256	4'hD	variable	1088 bit	512 bit

The cSHAKE128 and cSHAKE256 modes share the rate and capacity of SHAKE128 and SHAKE256 respectively; the sole datapath difference is the domain-separation suffix applied during final padding, which the core drives to 0x04 for cSHAKE in place of the 0x1F used for plain SHAKE. The function-name (N) and customization (S) strings are not parsed by the hardware. The customization preamble bytepad(encode_string(N) || encode_string(S), rate) defined in NIST SP 800-185 is the responsibility of the host: it is constructed externally and streamed ahead of the message, and the core absorbs the preamble and the message as one contiguous input. A cSHAKE mode selected with empty N and S is, by SP 800-185, equivalent to plain SHAKE; that case shall instead use the corresponding XOF_SHAKE128 or XOF_SHAKE256 mode. The hardware applies the cSHAKE suffix (0x04) unconditionally, so a cSHAKE mode driven without a preamble yields a digest that matches neither SHAKE (which pads with 0x1F) nor cSHAKE as defined for empty N and S.

Duet execution is selected implicitly rather than by a dedicated mode value: in the SHA-224 and SHA-256 modes (HASH_SHA2_224, HASH_SHA2_256) the 64-bit message bus always operates as two independent 32-bit lanes. The lane protocol, including how a single unpaired message is driven, is specified in Section 4.4.6. The mode encodings 4'hE and 4'hF are reserved and shall not be driven; the hardware decodes them as the SHAKE256 configuration, but this aliasing is an implementation artifact and shall not be relied upon.

3 Hardware Architecture

The main integration wrapper is `essec_sharmony_top`. It instantiates the padding front end (`essec_sharmony_pad`), the shared hash core (`essec_sharmony_core`), and the round-constant ROM (`essec_sharmony_rom`, 168×64 bit, holding both the SHA-2 and the SHA-3 constants), and exposes a streaming message input together with a streaming digest/XOF output. The SHA-2 round arithmetic uses dedicated adder modules whose implementation is selected by `ADDER_IMPL` (Section 3.3).

The design uses a single clock domain (`f_clk`) with a synchronous, active-low reset (`resetn`); clocking and reset behavior is specified in Section 3.2.

3.1 Block Diagram

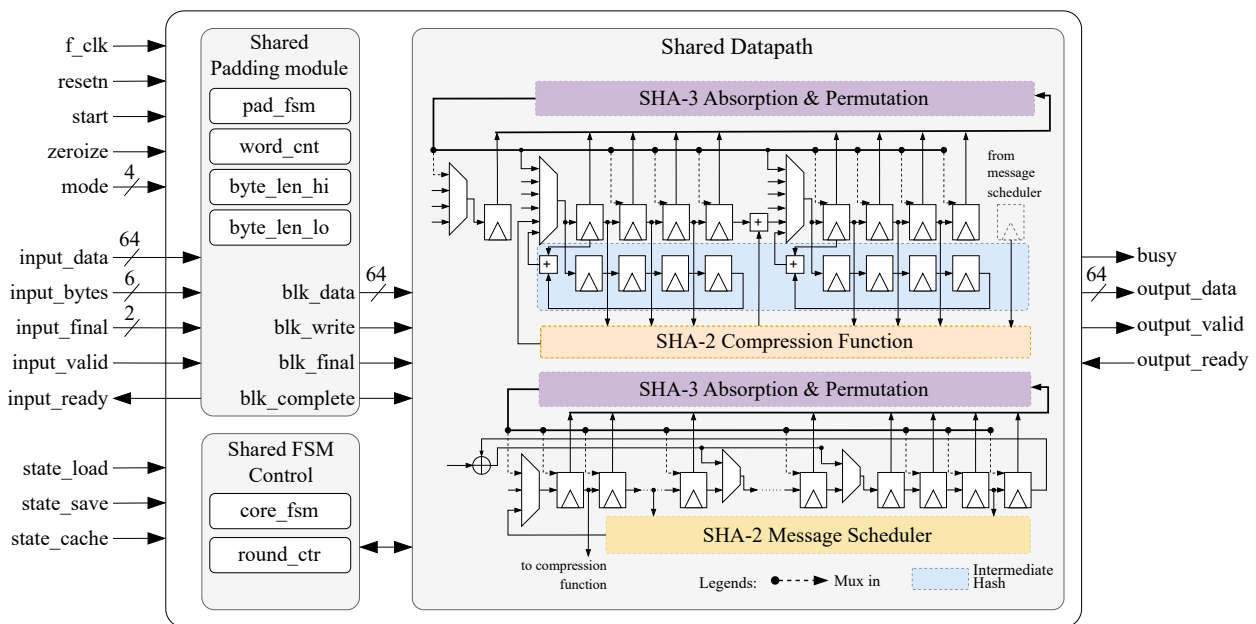


Figure 1: Top-level block diagram of SHARMONY: shared padding module, shared FSM control, and the shared SHA-2/SHA-3 datapath.

Detailed datapath schematics of the two halves of the shared register bank are provided in Appendix A (Section A.1).

3.2 Clocking and Reset

Clocking. The core operates in a single clock domain. All flip-flops are rising-edge-triggered on `f_clk`; the design contains no derived, gated, or inverted clocks and no clock-domain crossings. The implementation results in Section 5 were synthesized with a 10 ns (100 MHz) target clock constraint; the maximum achievable clock frequency is reported in Section 5.2.

Reset. `resetn` is active-low and strictly synchronous: it is sampled on the rising edge of `f_clk`, and every resettable flip-flop uses synchronous reset. The only registers without a reset are the read registers of the constants ROM, which never hold anything but fixed constant data. The clock must therefore be running while reset is applied, and `resetn` shall be held low for at least one rising clock edge. No post-reset wait cycles are required.

State after reset. Reset returns the padding and core state machines to idle and clears the shared 25×64 -bit state bank, the SHA-2 chaining registers, all counters, and the registered I/O boundary (`REG_I0 = 1`). At

the interface, `busy`, `output_valid`, and `input_ready` are deasserted; `input_ready` asserts only once a transaction has been started with `start`. The round-constant ROM holds fixed constants and has no resettable state.

Reset versus zeroization. `resetrn` is the hard reset: it clears the entire core, including interface and control state. `zeroize` is the soft, security-targeted mechanism that scrubs sensitive state in five cycles while control and interface remain operational (Section 4.4.9). Verified by `run-interface` (reset coverage) and `run-zeroize`.

3.3 Configuration Parameters

SHARMONY exposes three compile-time parameters on `essec_sharmony_top`: the booleans `REG_IO` (I/O-boundary registering, default 1'b1) and `ROM_USE_BRAM` (round-constants ROM implementation, default 1'b1), and the integer `ADDER_IMPL` (SHA-2 adder implementation, default `ADDER_CARRY4`). The defaults are defined in `essec_sharmony_pkg.sv` and may be overridden per instance. All combinations are functionally identical (same digests, same valid/ready handshake protocol) and differ only in I/O latency, port timing, and resource cost.

REG_IO (I/O boundary registering):

- `REG_IO = 1` (default). The control/data inputs and the digest/XOF outputs are registered at the module boundary. The internal datapath then has the full clock period available and the integrator sees only setup and clock-to-out paths at the ports. The trade-off is up to three additional cycles of I/O latency (`start` → first output) and approximately 271 extra flip-flops.
- `REG_IO = 0`. The boundary registers are omitted. This eliminates the I/O-register latency and the associated flip-flops (and a few LUTs), but it exposes combinational paths of roughly 5–6 ns at the ports, including a direct input-to-output path; the integrating system must close these paths within its own timing budget.

ROM_USE_BRAM (round-constants ROM implementation):

- `ROM_USE_BRAM = 1` (default). The 168×64 -bit SHA-2/SHA-3 round-constants ROM maps to a single block RAM tile (`rom_style="block"`); the registered read is provided by the tile's own output register.
- `ROM_USE_BRAM = 0`. The ROM is mapped into fabric LUTs (`rom_style="distributed"`), releasing the block RAM tile at a cost of about +310 LUTs and +65 flip-flops (the output register now uses explicit fabric flip-flops). Timing is unaffected. This configuration is intended for designs in which block RAM is the scarce resource.

The address layout of the shared 168×64 -bit ROM, with the per-mode base offsets, is shown in Figure 16 (Appendix A.1).

ADDER_IMPL (SHA-2 round-adder implementation):

- `ADDER_GENERIC` (0). Portable RTL + adders; used for technology-independent ASIC mapping (see Section 5).
- `ADDER_CARRY4` (1, default). Xilinx CARRY4 carry-chain primitives for the SHA-2 round arithmetic on 7-series FPGAs.
- `ADDER_CARRY8` (2). CARRY8 primitives for UltraScale and UltraScale+ carry chains.

3.4 Requirements

Table 2: Hardware requirements and their verification approach.

Req. ID	Requirement	Priority	Verification
REQ-HW-001	The module shall implement SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, and SHA-512/256 per FIPS 180-4.	Mandatory	Test
REQ-HW-002	The module shall implement SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, and SHAKE256 per FIPS 202.	Mandatory	Test
REQ-HW-003	The module shall implement the cSHAKE128 and cSHAKE256 modes with the SP 800-185 domain-separation suffix and the SHAKE128/SHAKE256 rate and capacity, absorbing the host-supplied, bytepad-encoded function-name (N) and customization (S) preamble as message input (Section 2.2).	Mandatory	Test
REQ-HW-004	The module shall support concurrent two-lane SHA-224/SHA-256 duet execution on the split 64-bit datapath.	Mandatory	Test
REQ-HW-005	The message input and digest/XOF output streams shall implement valid/ready handshaking with full back-pressure support.	Mandatory	Test
REQ-HW-006	The module shall support mid-state export and re-import (state_save/state_load) for all modes, and on-chip mid-state caching (state_cache) for SHA-2.	Mandatory	Test
REQ-HW-007	On assertion of zeroize, all sensitive registers shall be scrubbed within a fixed five-cycle window; the input stream shall not be accepted while zeroization is active.	Mandatory	Test
REQ-HW-008	The module shall not accept message beats presented before start, shall ignore start pulses driven during an active operation or output drain, and shall not emit spurious output in response to any refused access; unsupported control combinations (such as state_cache in a SHA-3 mode) shall complete without effect. The module shall not hang under any of these violations.	Mandatory	Test

4 Interfaces and Connectors

4.1 Interface List

The top-level interface of `essec_sharmony_top` consists of five groups: clock and reset, operation control, state management, and the two streaming data interfaces. Table 3 summarizes the groups; Table 4 specifies the individual signals.

Table 3: Top-level interface groups.

ID	Interface	Type / Standard	Description
IF-01	Clock / reset	Clock; level signal	f_clk, resetn; single clock domain, synchronous active-low reset.
IF-02	Operation control	Level/pulse signals	start, mode, zeroize, busy.
IF-03	State management	Level signals, sampled with start	state_load, state_save, state_cache.
IF-04	Message input stream	64-bit valid/ready	Streaming message input with back-pressure (input_*).
IF-05	Digest/XOF output stream	64-bit valid/ready	Streaming digest or XOF output with back-pressure (output_*).

4.2 Top-Level Signal List

Table 4 specifies the ports of essec_sharmony_top.

Table 4: Top-level signal list of essec_sharmony_top.

Signal	Direction	Width	Description
f_clk	Input	1	Clock input. All state updates occur on the rising edge of this clock.
resetn	Input	1	Synchronous active-low reset. Hold low for at least one clock cycle; reset behavior and the state after reset are specified in Section 3.2.
start	Input	1	Starts a new hash/XOF operation. Assert when busy is low and keep mode stable for the selected operation. A one-cycle pulse is sufficient; pulses asserted while busy is high are ignored.
zeroize	Input	1	Requests zeroization of internal state. A one-cycle pulse is sufficient. While zeroization is active, the input stream is not accepted and busy stays high.
mode	Input	4	Selects the hash/XOF algorithm (mode_e). Use one of the valid values from the supported modes table; 4'hE and 4'hF are reserved (decoded as the SHAKE256 configuration) and shall not be driven.
busy	Output	1	Busy status. High while a hash/XOF operation or zeroization is in progress (padder or core non-idle). A new start is accepted only when busy is low.
state_load	Input	1	Context switching (mid-state load). When high at start, the operation resumes from a loaded state: the first N input beats are the saved mid-state (SHA-2: 8 beats; SHA-3: 25 beats) and replace the IV / zeroed sponge. Sampled with start and held for the operation.
state_save	Input	1	Context switching (mid-state save). When high at start, the terminal beat (input_final) exports the full un-padded internal state (SHA-2: 8-word chaining value; SHA-3: 25-lane KECCAK state) instead of the padded digest. Requires block-aligned input. Sampled with start.
state_cache	Input	1	On-chip mid-state fast path (SHA-2). When high at start together with state_save/state_load, the mid-state is kept in / resumed from an internal single-slot register instead of being streamed over the message bus: no export beats on save, no re-streamed state beats on load. Lets a fixed prefix be computed once and reused across many message hashes without the off-chip round-trip. Sampled with start.
input_data	Input	64	Message input beat. Data is accepted only when input_valid and input_ready are both high.

Signal	Direction	Width	Description
input_valid	Input	1	Qualifies input_data, input_bytes, and input_final on the message input stream.
input_bytes	Input	6	Number of valid message bytes in the current input beat. For the 64-bit streaming modes, use values from 0 to 8. In the SHA-224/SHA-256 modes the field is always split into two 3-bit per-lane byte counts (0 to 4 each): [5:3] for the upper 32-bit lane and [2:0] for the lower 32-bit lane.
input_final	Input	2	Marks the final input beat. For the 64-bit streaming modes, the final beat must be marked with 2'b11. In the SHA-224/SHA-256 modes, bit 1 marks the upper 32-bit lane as final and bit 0 marks the lower 32-bit lane as final; the operation completes only when both lanes have finalized.
input_ready	Output	1	Message stream back-pressure signal. The wrapper accepts an input beat only when this signal is high together with input_valid. Deasserted while the engine is idle; asserts only after a transaction has been started with start.
output_data	Output	64	Digest or XOF output beat. The value is valid when output_valid is high. Note: SHA-2 fixed-length digests are streamed in reverse word order (most-significant word first); SHA-3/SHAKE stream in forward order. See the output word-order note in Section 4.3. Driven to zero whenever output_valid is low.
output_valid	Output	1	Qualifies output_data. For fixed-length hash modes, output words are consumed when output_valid and output_ready are both high.
output_ready	Input	1	Output consumer ready signal. For fixed-length hash modes, assert to consume digest words. For SHAKE modes, keeping this signal high continues XOF squeezing; deassert it to stop the XOF output stream.

4.3 Operation Sequence

1. Apply reset by holding resetn low for at least one clock cycle (reset is synchronous, sampled on the rising edge of f_clk), then release it high.
2. Wait until busy is low.
3. Drive the desired mode and pulse start high for one clock cycle.
4. Send message beats using the input valid/ready handshake: a beat is accepted when input_valid and input_ready are both high; use input_bytes to indicate the number of valid bytes in the beat; mark the last beat with input_final.
5. Read digest or XOF output beats using the output valid/ready handshake: a fixed-length digest beat is consumed when output_valid and output_ready are both high. For SHAKE modes, keep output_ready asserted for as many 64-bit output beats as required, then deassert it to terminate squeezing.
6. Wait for busy to return low before starting the next operation.

Input byte order. Message bytes fill each 64-bit input beat from the most-significant byte downward: byte k of a beat occupies bits $[63-8k : 56-8k]$, so when input_bytes < 8 the valid bytes are the uppermost bytes of the beat. In the SHA-224/SHA-256 modes the same convention applies per 32-bit lane (lane byte 0 in the lane's top byte, up to 4 bytes per lane). The convention is identical for every mode: the little-endian lane ordering of FIPS 202 is applied inside the core, so the host streams SHA-3/SHAKE messages exactly as it streams SHA-2 messages.

Output word order. SHA-2 fixed-length digests are emitted in **reverse word order**: the highest-index digest word first, down to D0 (e.g. D7, D6, ..., D0 for SHA-512; D6 ... D0 for SHA-224; D5 ... D0 for SHA-384; D3 ... D0 for SHA-512/224 and SHA-512/256). Each individual word is still in big-endian byte order. This lets the core stream the early-finalized words out during the last accumulation cycles (saving up to 4 output

cycles). SHA-3 and SHAKE are unaffected and stream in forward order (D0 first). A consumer must therefore reverse the SHA-2 word sequence to reconstruct the standard digest; back-pressure (`output_ready` low) is honored for every beat, including the overlapped ones. For SHA-512/224 and SHA3-224 (a 224-bit digest is 3.5 output words), digest word D3 is half-populated: its 32 valid bits occupy the upper half of the beat and the lower half is driven to zero.

4.4 Interface Timing Diagrams

This section specifies the cycle-level behavior of the streaming interface. The first two figures show complete, representative hash transactions (one SHA-2 and one SHA-3) from the accepted `start` through the final digest beat; the figures that follow isolate individual aspects of the protocol (the input and output handshakes, duet operation, XOF squeezing, invalid access, and zeroization). In every diagram the long internal bursts (block absorption, round computation) are gap-compressed so that the beginning and end of the transaction stay legible. Each figure is derived from a VCD capture of a test in the suite (`make run -<test> VCD=1`) and rendered as a WaveDrom¹ timing diagram.

4.4.1 Reading the Diagrams: the Registered I/O Boundary

All diagrams in this section are captured with the default `REG_IO = 1` configuration (Section 3.3), whose input boundary registers are visible in two places when reading the figures.

First, `busy` asserts on the same clock edge that samples an accepted `start` (or `zeroize`) pulse, while the FSM trace (`core_fsm_q`) leaves `IDLE` one cycle later. The `padder` and `core` are driven not by the external pulse but by its registered copy (`start_q` and `zeroize_q`, respectively, in `essec_sharmony_top`), which they see one cycle after the sampling edge; `busy` intentionally covers this input-register stage so that no cycle exists in which a command has been accepted but the engine still reports `idle`. The one-cycle overlap of `busy` high with `IDLE` is therefore the signature of the registered command path, not a stall; under `REG_IO = 0` the two transitions coincide.

Second, the meaning of `input_ready` differs between the two configurations. With `REG_IO = 1` a one-beat buffer sits between the host and the `padder`, and `input_ready` high means this buffer has room, not that the `padder` is streaming: the buffer can take the first message beat as early as one cycle after `busy` rises and hold it until the `padder` is ready to consume it. With `REG_IO = 0` there is no buffer, and `input_ready` high means the `padder` is consuming beats right now, which is several cycles later (in the SHA-3 modes, only once `SHA3_INIT` has completed). In both configurations the host simply follows the handshake: a beat is transferred when `input_valid` and `input_ready` are both high, and no fixed cycle count between `start` and the first accepted beat is guaranteed by this specification.

4.4.2 SHA-512 Hash Transaction

Once `start` is accepted with `busy` low, the message is streamed and marked final. The figure shows the shortest possible transaction, the empty message (the first NIST CAVP ShortMsg vector): a single beat with `input_bytes = 0` and `input_final = 2'b11`. A zero-byte final beat is legal; it carries no data and only delivers the end-of-message marker, from which the `padder` builds the complete padding block with no further host involvement.

The core then runs the SHA-2 compression (`INIT` → `ROUNDS`, with the round burst gap-compressed in the figure) and emits the eight digest words on `output_data` under the output handshake. With the default `REG_IO = 1`, the digest does not leave the core directly: it passes through a registered output boundary (a collection register feeding the bus-facing output register, with a one-deep skid buffer between them). The first digest words enter this boundary while the compression is still finishing: the working variables e through h form a shift chain, so the four E-chain digest words (D7 down to D4) become final one per cycle

¹<https://wavedrom.com>

over the last four round cycles and are emitted as they do (the chain is drawn in the datapath schematic of Figure 15, Appendix A.1). The digest stream is therefore aligned to the output handshake rather than to the FSM trace; Section 4.4.5 shows both consequences under back-pressure. SHA-2 digests leave the core in reverse word order (D7...D0); see Section 4.3.

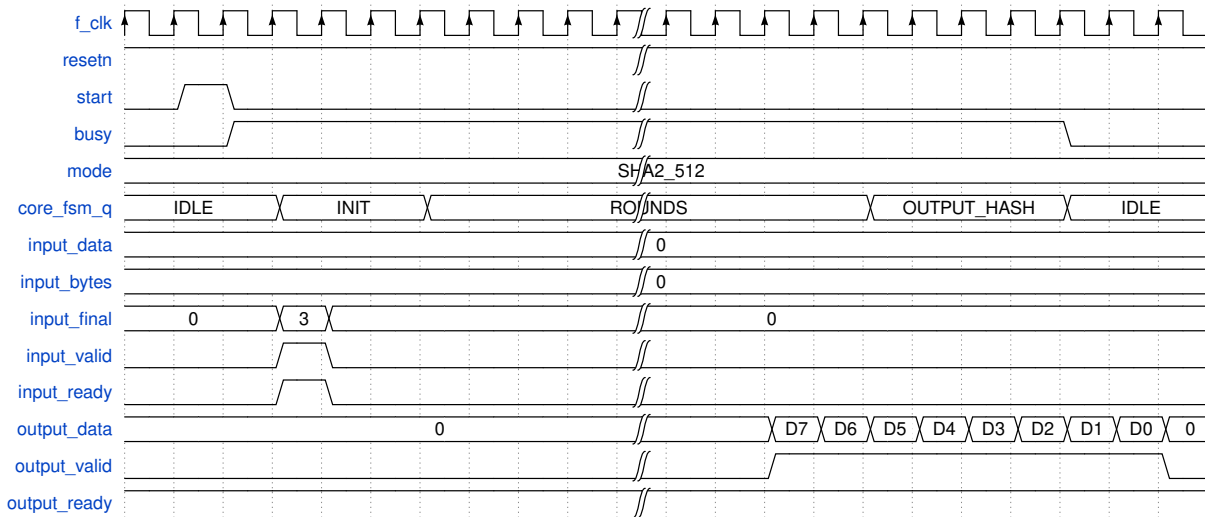


Figure 2: SHA-512 hash of the empty message.

4.4.3 SHA3-256 Hash Transaction

The sponge path inserts the SHA3_INIT and ABSORB phases ahead of the K_{ECCAK} permutation: SHA3_INIT clears the 25-word state bank in five cycles through the same shift chains that the zeroization scrub uses (Section 4.4.9), and ABSORB XORs the padded block into the rate portion of the state (17 of the 25 lanes for SHA3-256, 1088 bits), one 64-bit lane at a time. Both bursts are gap-compressed in the figure. The transaction hashes the empty message: a single zero-byte final beat, after which the padder builds the padded block.

The four digest words are emitted in forward word order (D0...D3), in contrast to the reversed SHA-2 order. There is also no early emission here: unlike SHA-2, whose digest words become final one per cycle at the end of the compression, the SHA-3 digest is simply the leading lanes of the state, and no lane is final until the last permutation round completes. The core instead taps the final round's combinational output directly, loading the first word into the output boundary's collection register on the same clock edge that finishes the permutation; one register stage later D0 is on the bus, so output_valid rises one cycle into OUTPUT_HASH, as the figure shows. The remaining words stream from the state bank one lane per accepted beat.

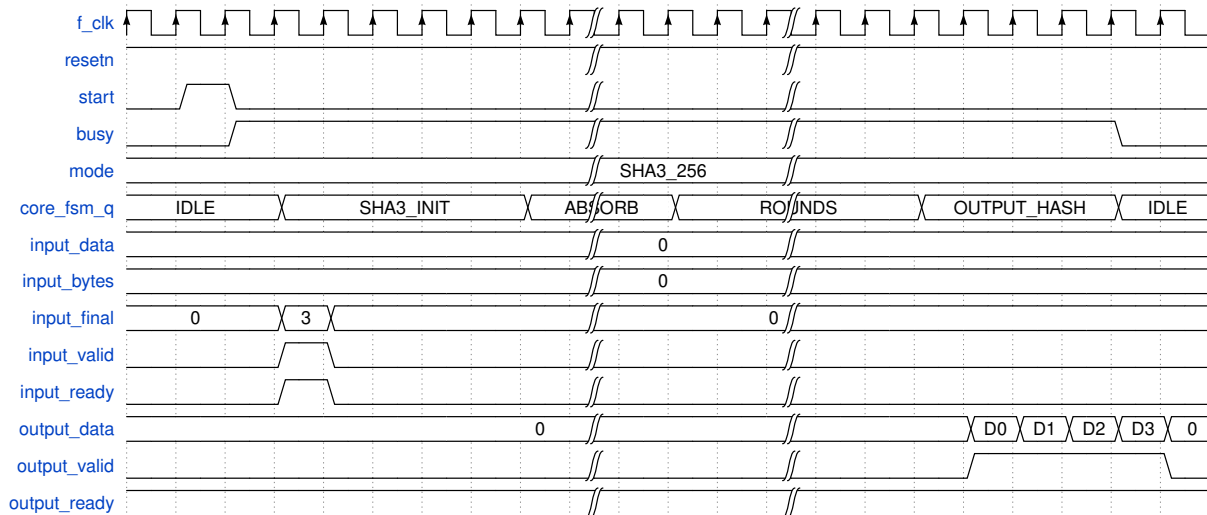


Figure 3: SHA3-256 hash of the empty message.

4.4.4 Message-Input Handshake

A message beat is consumed only when `input_valid` and `input_ready` are both high. Withholding `input_valid`, or `input_ready` going low while the core absorbs a block, stalls the stream with no loss of data. The initial `input_ready` pulse in `WAIT_CORE` is the one-deep input buffer of Section 4.4.1 accepting an early beat before streaming begins. The lower traces show the padder consuming the stalled stream: accepted beats (`pad_accept_msg`) are forwarded as block words B0-B3 (`pad_blk_write`), and after the final beat the padder appends the 0x80 marker (80_HI), the zero fill (ZEROS, compressed), and the 128-bit length field (LEN, word B5) with no further host involvement. The window ends with `busy` still high; compression and digest output proceed as in Figure 2. Verified by `run-input_handshake`.

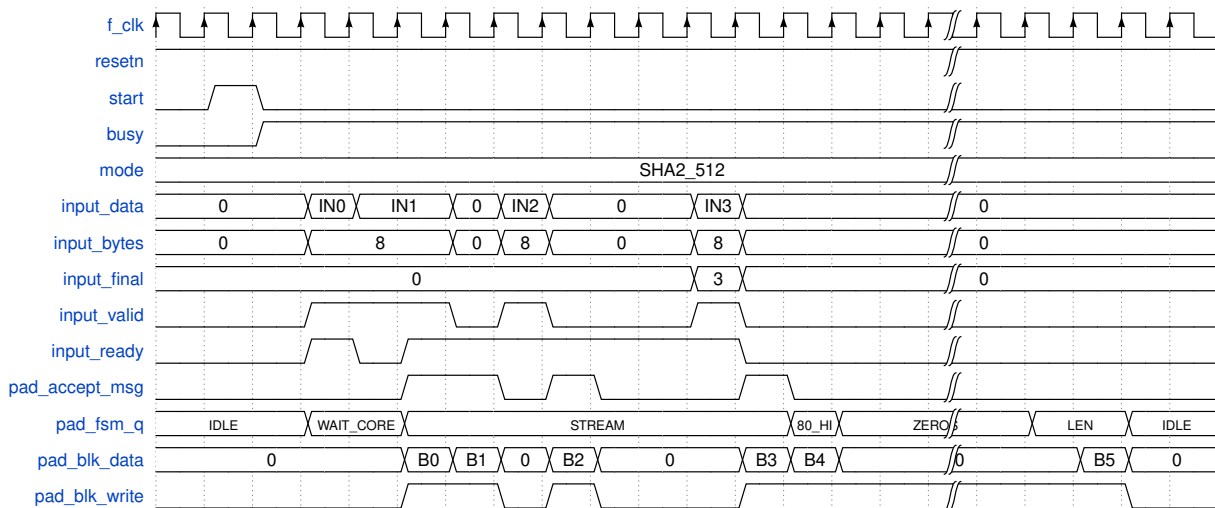


Figure 4: Message-input valid/ready handshake with stalls.

4.4.5 Output Back-Pressure

Each digest beat is held with `output_valid` asserted until the consumer raises `output_ready`, so back-pressure is honored on every beat. The digest stream is aligned to the handshake, not to the FSM trace: the registered output boundary of Section 4.4.2 decouples the core from the bus at both ends of the drain. At the head, the core loads the E-chain words during the final round cycles, so D7 appears while `core_fsm_q`

still shows ROUNDS. At the tail, the core returns to IDLE and busy deasserts once the last word has been handed to the boundary; at that point the boundary may still hold up to three undrained beats, and the final beats (D1 and D0 in the figure) drain after busy is already low. A low busy therefore does not mean the digest has been consumed: the host reads the complete digest before starting the next operation, in the order the operation sequence of Section 4.3 prescribes. Verified by run-output_handshake.

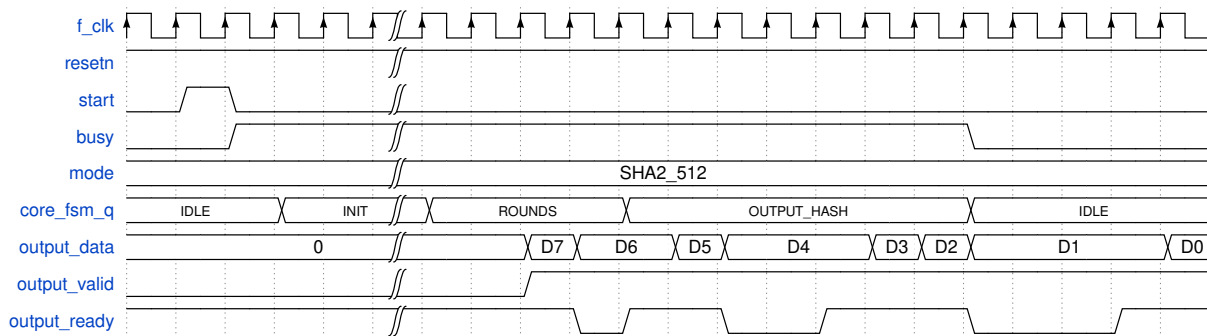


Figure 5: Digest-output handshake under back-pressure.

4.4.6 Duet Execution

In the SHA-224/SHA-256 modes the two 32-bit lanes run independently on the split 64-bit bus: `input_data[63:32]/input_bytes[5:3]/input_final[1]` carry the high lane and `[31:0]/[2:0]/[0]` the low lane. A transaction completes only after both lanes have signaled final. Both lanes shall require the same number of padded message blocks: for a lane message of L bytes, the padded block count is $\lceil (L + 9)/64 \rceil$, and the two lane lengths must yield the same count (for example, both messages of at most 55 bytes, or both of 56 to 119 bytes). Within this shared block schedule the lanes are otherwise independent: they may carry different message lengths and byte counts and may finalize on different beats, including on the same beat with different per-lane byte counts. The restriction follows from the lockstep datapath: the two lanes advance through the single shared compression FSM together, and a lane has no per-lane freeze once finalized. If the block counts differ, the transaction still completes and does not hang, and the lane with the larger block count produces its correct digest, but the other lane's digest is undefined and shall not be used. A host holding two messages with different block counts hashes them as separate transactions instead. Verified by run-duet.

Two concurrent messages are hashed by streaming one per lane. A host with a single unpaired message, for example the leftover node when a Merkle-tree level with an odd number of nodes is hashed in sibling pairs, drives it identically on both lanes: the same data, byte count, and final flag on both halves of each input beat. The two lanes then compute the same digest, and the host reads it from either lane. In the figure the high lane finalizes on the first beat (`input_final[1]`) and then idles while the low lane keeps streaming and finalizes one beat later (`input_final[0]`). A finalized lane simply presents `input_bytes = 0` on subsequent beats; the zeroed high-lane fields during the second transfer denote an idle lane, not a zero-data message beat. Both lanes are served by the single shared compression datapath (`core_fsm_q`: one INIT $\rightarrow$ ROUNDS pass for the pair), after which the two digests are emitted on the same split bus (`output_data[63:32]` for the high lane, `[31:0]` for the low lane) under the `output_valid/output_ready` handshake, with two beats held under one and two cycles of back-pressure. Each output beat carries the same-index digest word of both lanes, in the same reverse word order as solo SHA-2 output: eight beats (D7...D0) for SHA-256 and seven beats (D6...D0) for SHA-224. Verified per lane by run-duet.

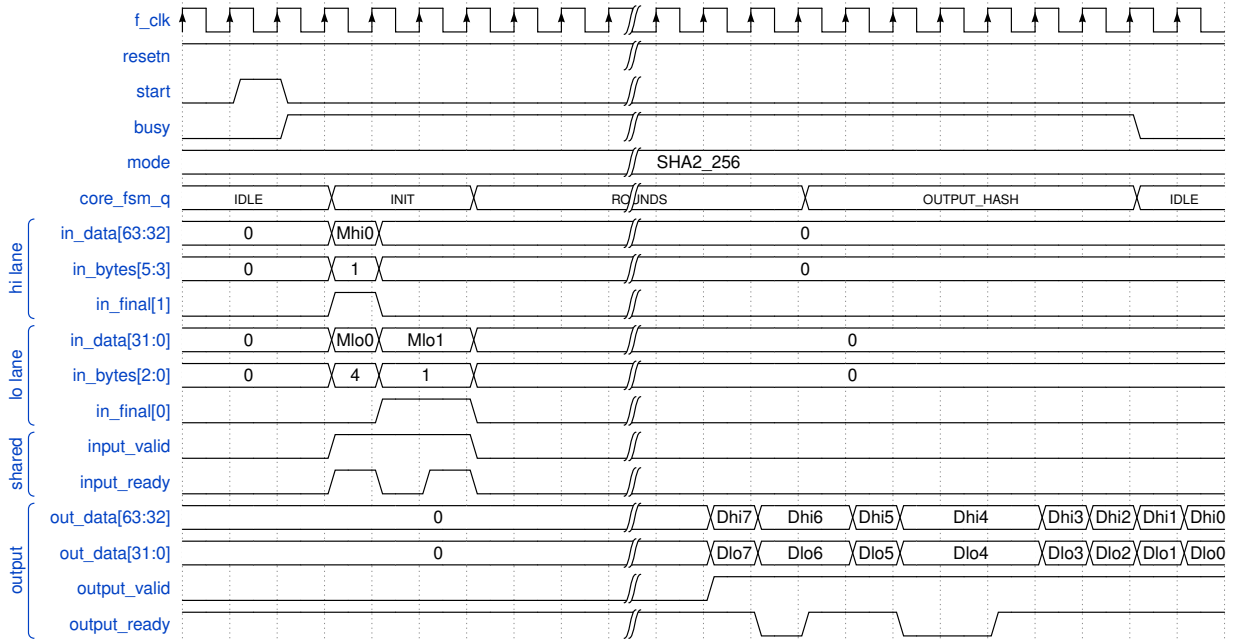


Figure 6: Duet execution: two independent SHA-256 hashes run concurrently, one per 32-bit lane of the split 64-bit bus, finalizing independently and emitting both digests on the shared output handshake.

4.4.7 XOF Squeeze

The figure opens mid-transaction, during message absorption; the `start` pulse and input streaming are as in the preceding figures. After the message is absorbed and the initial KECCAK permutation runs (`core_fsm_q`: ABSORB → ROUNDS), the engine squeezes output beats (OUTPUT_XOF, forward word order Z0, Z1, ...). Once a full rate has been squeezed (1088 bit = 17 beats, Z0...Z16), a second permutation runs automatically (WAIT → ROUNDS, with `output_valid` momentarily low) before squeezing resumes (Z17, ...). The output length is set by the consumer: holding `output_ready` high continues the squeeze, and deasserting it terminates the stream. The final beat need not be acknowledged: in the figure the consumer samples Z19 while it is presented and withholds `output_ready`, so the engine stops without consuming it. After the stop the engine returns to IDLE and `busy` deasserts, signaling that a new operation may be started. With `REG_IO = 1` the stop leaves the output boundary holding the offered beat and up to two staged look-ahead words; they remain retrievable through the handshake while the engine is idle and are cleared by the next `start`, `zeroize`, or `reset`. A host that requires this residue to be destroyed shall issue `zeroize` (Section 4.4.9). With `REG_IO = 0` no residue exists: `output_valid` falls as the engine stops. Verified by `run-shake256_variableout`.

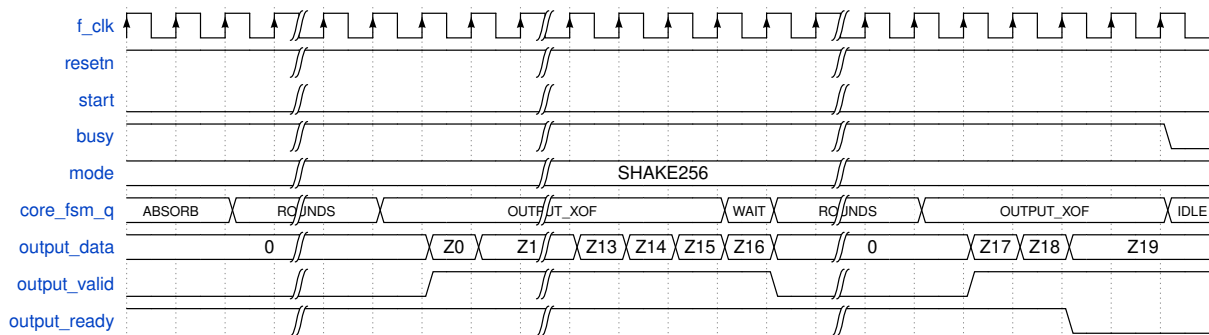


Figure 7: SHAKE256 extendable-output (XOF) squeeze; the consumer stops the stream by withholding `output_ready`, leaving the final beat (Z19) presented but unconsumed.

4.4.8 Invalid Access

Illegal traffic is a complete no-op: it is refused at the boundary and leaves no residual state behind. The figure demonstrates this at the three most sensitive moments of a transaction. First, a data beat presented before start (left edge) is refused: `input_ready` stays low, the beat is not accepted, and no spurious `output_valid` is produced. Second, a start pulse driven in the middle of the computation, together with a hostile mode change, is ignored: a start is only accepted while `busy` is low, so neither the round computation nor the latched mode is disturbed. Third, another start pulse fires while the digest is still draining; the one-cycle `output_ready` dip in the same window is back-pressure applied by the host, not a reaction of the engine; the pending beat (D5) is simply held stable until `output_ready` returns, as the stream contract requires, and the drain continues with no beat lost or duplicated. The transaction completes with a bit-exact digest, proving that none of the three violations left any trace. Verified by run-error.

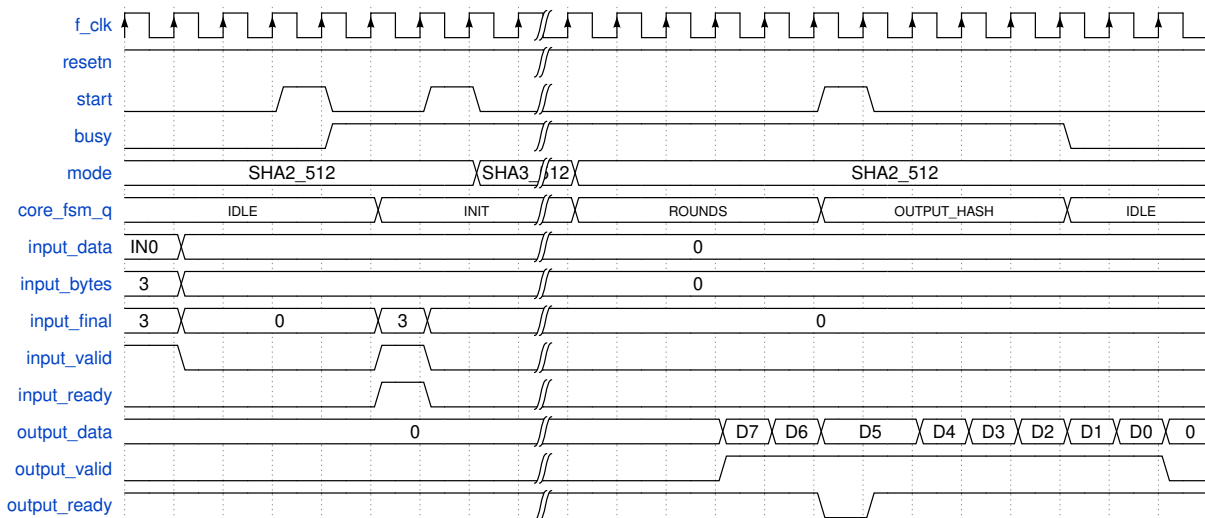


Figure 8: Invalid-access handling: a refused pre-start beat, an ignored restart with a hostile mode change during computation, and an ignored restart during the output drain under host back-pressure.

4.4.9 Zeroization

`zeroize` is sampled like any other command: a single-cycle pulse is sufficient, `busy` asserts on the sampling edge (already high in the figure, where the scrub interrupts a running transaction), and it remains high until the scrub has completed; message input is refused throughout (`input_ready` low). Internally, `zeroize_sub` is held for `ZEROIZE_IDX+1` cycles while the core sits in the `ZEROIZE` state. The scrub drains the sensitive banks through their existing shift chains rather than broadcasting a clear to every register: a zero is injected at each chain head on every cycle of the window and marches through the chain one register per cycle. The figure makes this visible as a staircase: `R_q[8]` down to `R_q[4]` (at five registers the longest scrub segment, and therefore the chain that sets the window length), and `H_q[0]` down to `H_q[3]` each clear exactly one cycle after their predecessor (NZ marks a nonzero value). `R_q[4]` is the last register in the engine to clear, one cycle before `busy` falls; the digest chain additionally shows the mirrored staircase at the start of the transaction as it fills. Note also that the digest held in the output register (`output_valid` high, first beat held) is destroyed by the scrub: `output_valid` falls and `output_data` clears. Only after every chain has flushed does the core return to `IDLE` and `busy` deassert. The figure shows two representative chains; all `H_q/R_q` registers and the padder input registers are scrubbed. Verified by run-zeroize and the `zeroize_scrub` sweep.

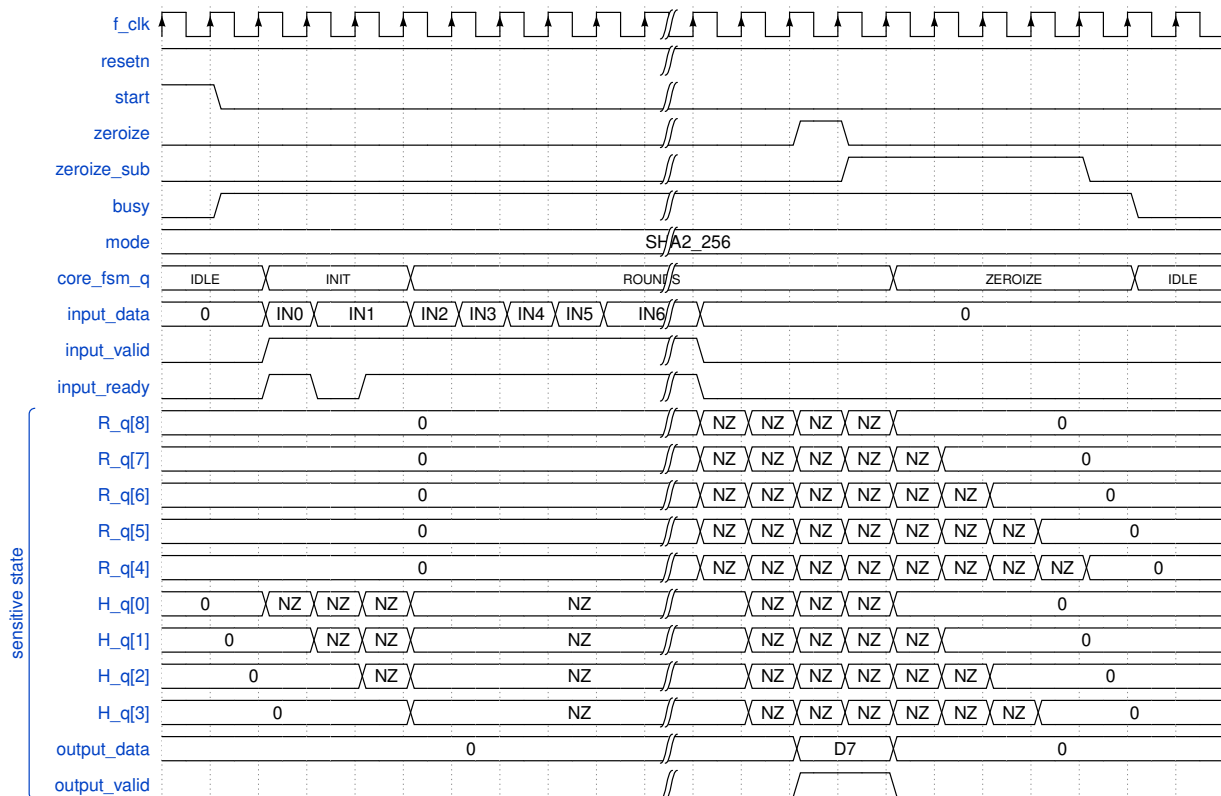


Figure 9: Zeroization: the zero shift marches through the scrub chains one register per cycle (`R_q[8..4]` spans the full five-cycle window), scrubbing the pending digest on the way.

4.5 State Management

The `state_load`, `state_save`, and `state_cache` inputs are all sampled together with `start` and determine how the operation is initialized and how its terminal beat is handled. Two complementary use models are supported:

- **Context switching** (`state_save / state_load`). The un-padded internal state is exported and later re-imported, allowing a hash context to be suspended, resumed, or interleaved with others. This mechanism underlies SHA context switching and keyed constructions built on it, such as HMAC and KMAC.
- **Mid-state caching** (`state_cache`). A computed prefix state is held in an on-chip single-slot register and reused across many message hashes with no bus transfer, targeting fixed-prefix PQC hashing (SLH-DSA, XMSS).

4.5.1 Context Switching (state_save / state_load)

In this mode the internal state is transferred in and out over the 64-bit message bus:

Table 5: State-transfer operations selected by state_load and state_save.

state_load	state_save	Operation	Starts from	On the terminal beat
0	0	Normal hash	IV / zeroed state	pad → emit digest
0	1	Compute + save	IV / zeroed state	no pad → export mid-state
1	0	Resume + finalize	loaded mid-state	pad → emit digest
1	1	Resume + save	loaded mid-state	no pad → export mid-state

- **Mid-state size.** The transferred state is 8×64 -bit beats for SHA-2 (the chaining value H ; in the SHA-224/SHA-256 modes each beat carries the same-index chaining word of both lanes, high lane in bits [63:32], low lane in [31:0], as on the data bus) and 25×64 -bit beats for SHA-3/SHAKE (the complete KECCAK state).
- **Load.** When `state_load` is asserted, the N mid-state beats are streamed ahead of the message, in the identical order and format that `state_save` emitted.
- **Save.** The input must be block-aligned, since no padding is appended; the state exported this way re-imports byte-for-byte through `state_load`. The last input beat is still marked with `input_final`: with `state_save` asserted, the marker ends the input stream without appending padding. This applies to the cached save flow as well.
- **Resume and finalize.** The SHA-2 length field assumes a single-block prefix, so the appended length is the *total* length of the prefix plus the resumed message.
- Because the complete state is moved, this path applies to every mode (including SHA-3/SHAKE) and provides the basis for HMAC, KMAC, and other resumable, context-switched hashing.

4.5.2 Mid-State Caching (`state_cache`)

For SHA-2, `state_cache` keeps the prefix mid-state entirely on-chip instead of moving it over the message bus. The cache is a single slot that reuses the working chaining registers `H_q` directly, so it adds no dedicated storage:

- **Cache-save** (`state_cache` & `state_save`, sampled at `start`). The block-aligned prefix is absorbed and its chaining value is retained in `H_q`. Unlike a streamed save, nothing is emitted on `output_data`: the mid-state never leaves the device (Figure 12). Output silence on this path is a checked invariant of the regression, so a mid-state leak during a cache-save is a test failure rather than a latent defect.
- **Cache-resume** (`state_cache` & `state_load`, sampled at `start`). The operation restarts directly from the resident `H_q`: no IV is loaded and no state beats are re-streamed on `input_data`. Only the message tail is applied, after which the engine finalizes to a digest (Figure 13). The cached mid-state survives each completed resume, so a single cache-save supports any number of subsequent resumes (verified by the multi-resume scenario of `run-midstate`). A `state_save` asserted together with a cache-resume is ignored at the sampling point: the resume wins.

A prefix is therefore hashed once and reused across many message hashes at zero per-message transfer cost: the building block for fixed-prefix PQC hashing, such as the repeated PK . seed prefix in SLH-DSA or the SEED prefix in XMSS. Because the cache is exactly the working `H_q` registers, `zeroize` destroys the cached mid-state together with the rest of the sensitive state (the scrub of Figure 9); there is no separate retained copy that can outlive a scrub.

Caching covers SHA-2 only, including the SHA-224/256 dual-lane (duet) case: both lanes' chaining values occupy the same 64-bit `H_q` words and are cached and resumed together, and the lanes may carry different messages and lengths within the same padded block count (Section 4.4.6; verified by the duet-cache sweeps of `run-midstate`). Asserting `state_cache` in a SHA-3/SHAKE mode is safe but has no caching effect: a cache-save is a silent no-op, and a cache-resume degrades to a plain hash of the streamed message, without hanging the engine (verified by `run-sha3cache`). When `state_cache` is low, save and load use the streamed context-switching flow above, which remains the only path for SHA-3/SHAKE.

4.6 State-Management Timing

Figures 10 and 11 show the two halves of one streamed save/resume round-trip (SHA-512, same run): the mid-state exported by the first transaction is re-injected byte-for-byte as the input of the second. SHA-3/SHAKE use the same protocol with a 25-word state. Verified by `run-midstate`.

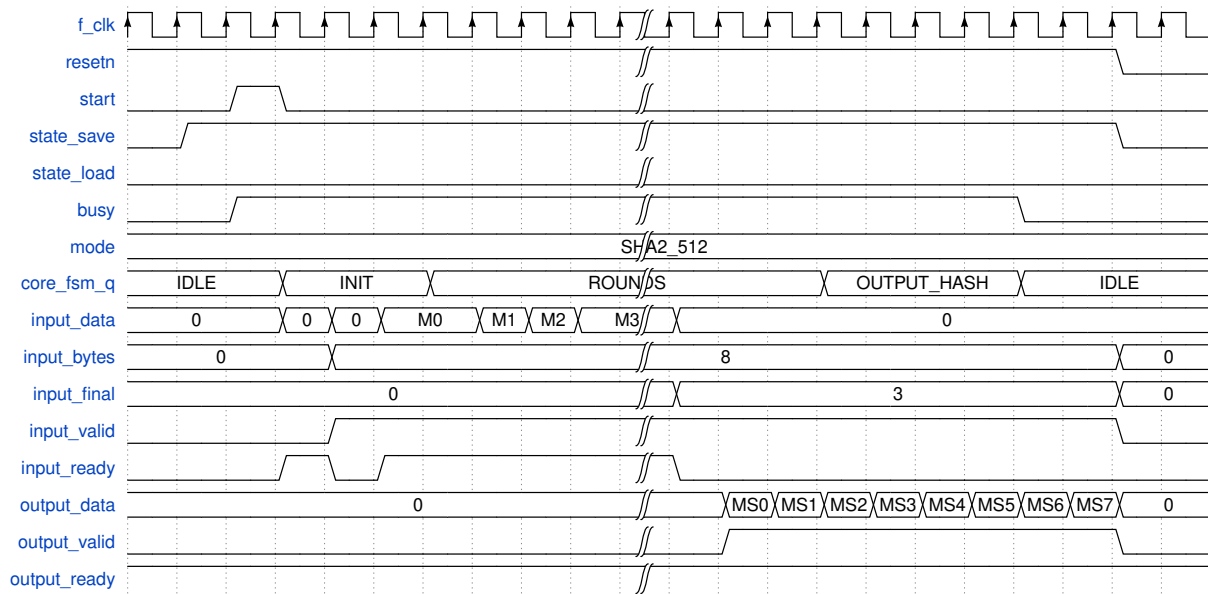


Figure 10: Mid-state save (SHA-512). With `state_save` qualified at start, a block-aligned message (M0...M15) is absorbed and, instead of padding to a digest, the full 8-word chaining state is exported on `output_data` (MS0...MS7, no padding). The round burst is gap-compressed.

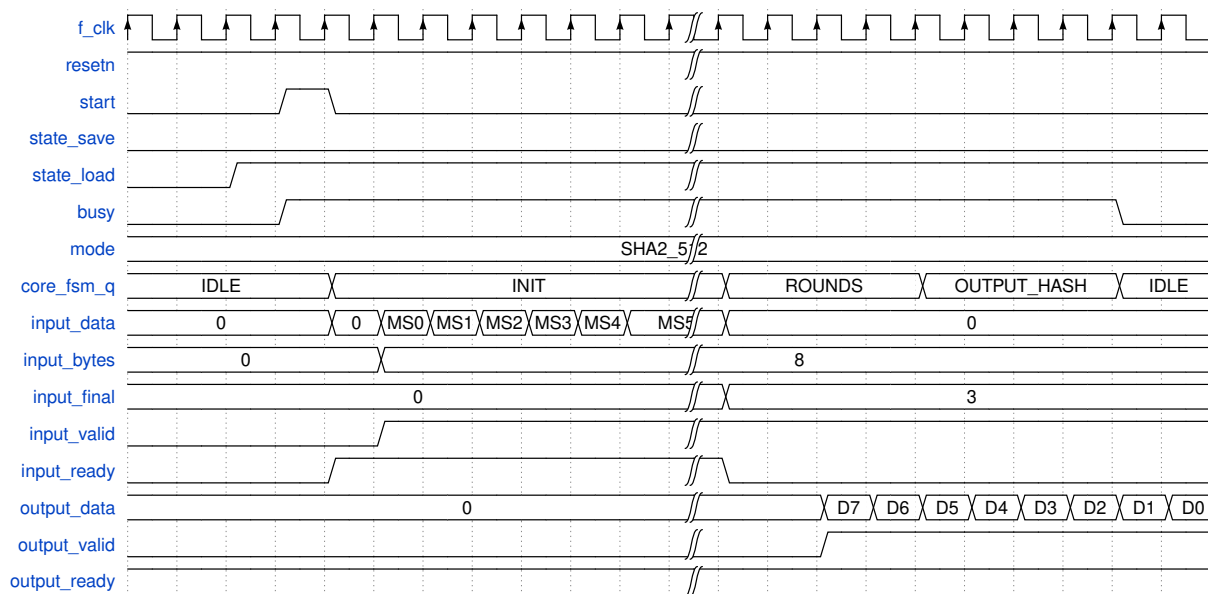


Figure 11: Mid-state resume (SHA-512). With `state_load` qualified at start, the eight saved words are re-injected as the first input beats (MS0...MS7, `input_final` low), followed by the message tail (absorbed within the compressed region); the operation then finalizes to the digest (D7...D0, SHA-2 reverse word order).

The on-chip cache path (`state_cache`) avoids the state transfer entirely. Figures 12 and 13 show the same prefix cached and then resumed (SHA-512): the `H_q` row group carries the chaining value (shown as MS while populated), and it stays resident across the whole round-trip while neither bus moves the state. Verified by `run-midstate_cache`.

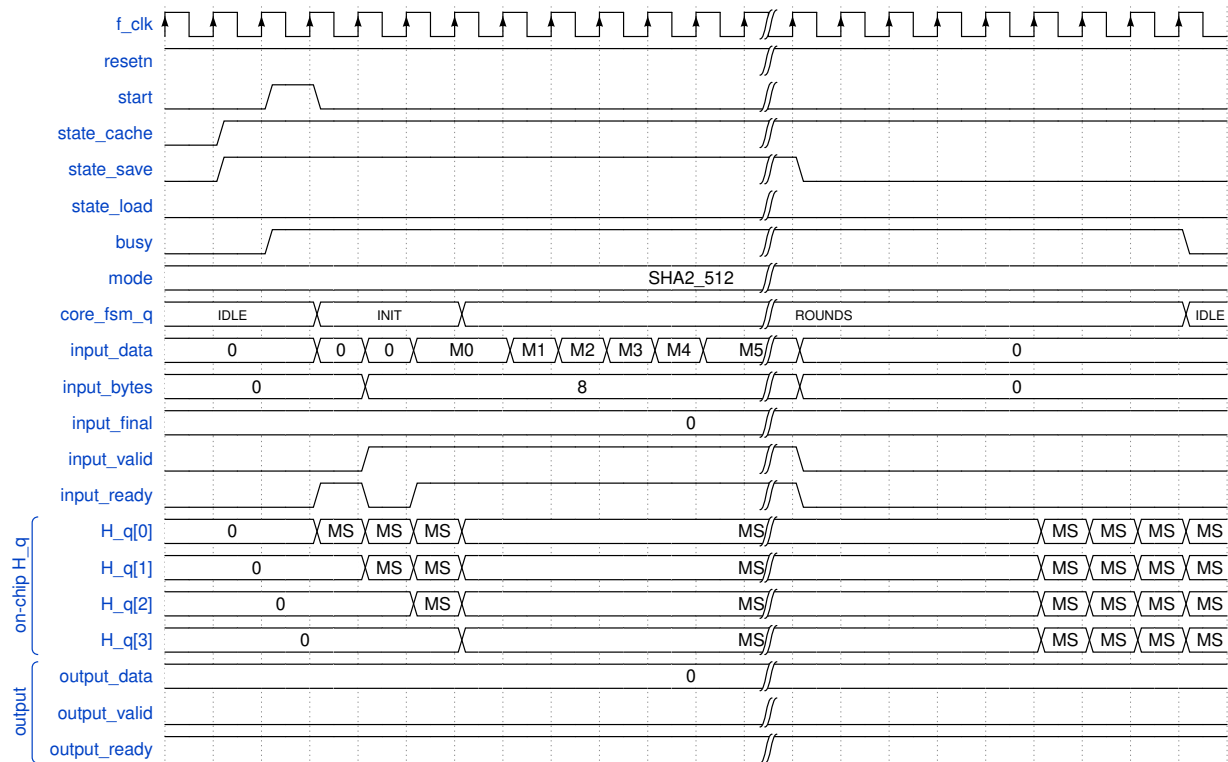


Figure 12: Cache-save (SHA-512, state_cache with state_save). The block-aligned prefix (M0...) is absorbed and its chaining value forms in the on-chip H_q registers, which remain populated (MS) after busy falls. In contrast to the streamed save of Figure 10, output_data stays idle throughout: the mid-state is retained on-chip and never appears on the bus. The round burst is gap-compressed; the final prefix beat, marked with input_final, lies inside the compressed region.

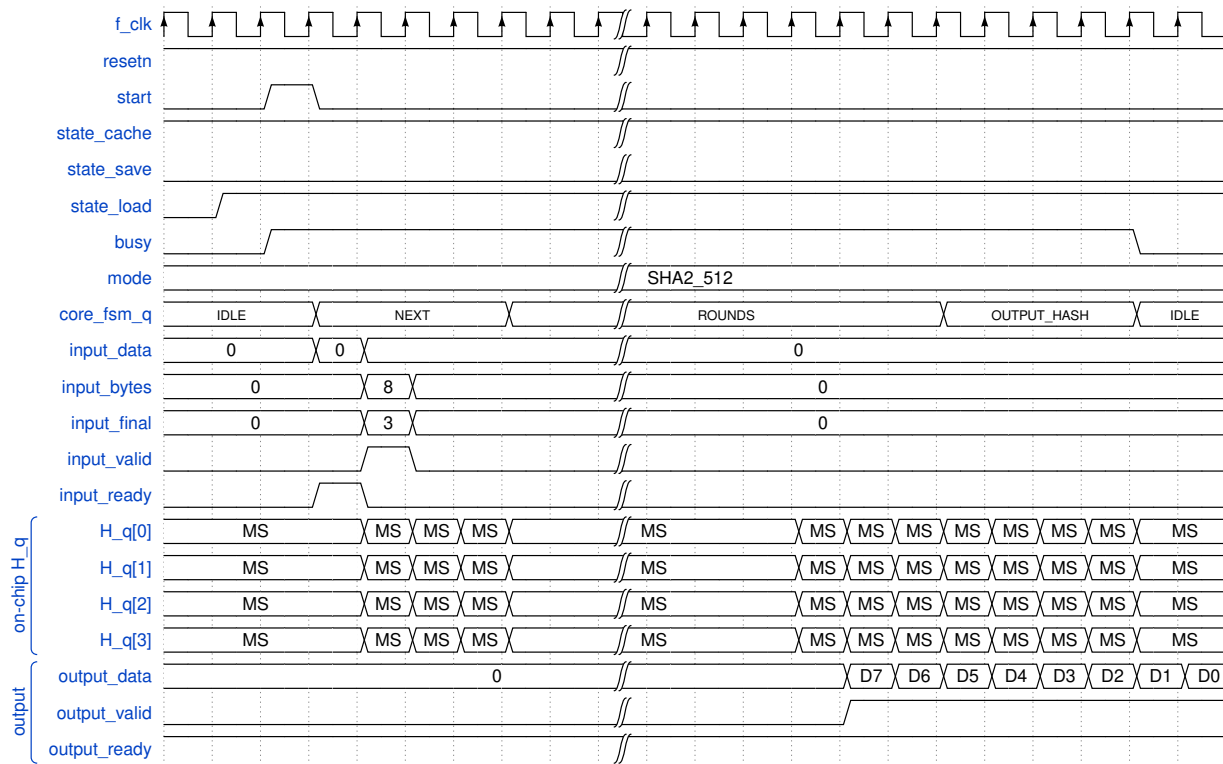


Figure 13: Cache-resume (SHA-512, state_cache with state_load). The engine restarts from the resident H_q (already MS at start): no state beats are re-streamed on input_data; only the message tail is applied, and the operation finalizes to the digest (D7...D0, SHA-2 reverse word order). state_cache remains high throughout, held from the preceding save. Contrast the streamed resume of Figure 11, which re-injects MS0...MS7 on the input bus.

5 Implementation Results

The results in Sections 5.1 and 5.2 are from Vivado out-of-context synthesis for xc7a200t fbg484 - 3 (Artix-7, speed grade -3) with a 10 ns clock constraint and the default synthesis directive. The I/O ports carry no input/output-delay constraints, so the boundary delays are the measured worst-case combinational path delays, and the internal Fmax reflects the register-to-register critical path (the SHA-2 round chain: the $\Sigma_1/\text{Ch}/\text{adder}$ path updating E), which is identical for both configurations. All flip-flops use synchronous reset and map to the FDRE primitive. Neither configuration uses DSP slices.

5.1 FPGA Resource Utilization

The utilization in Table 6 is reported for the default ROM mapping (ROM_USE_BRAM = 1). The distributed ROM mapping (ROM_USE_BRAM = 0) instead releases the block RAM tile and adds approximately 310 LUTs and 65 flip-flops.

Table 6: Post-synthesis FPGA resource utilization (Artix-7, xc7a200t fbg484 - 3).

Resource	REG_IO = 1 (default)	REG_IO = 0
Slice LUTs (total)	5873	6044
— LUT as logic	5745	5916
— LUT as memory (SRL16E)	128	128

Resource	REG_IO = 1 (default)	REG_IO = 0
Slice registers (FF, all FDRE)	2310	2039
CARRY4	254	254
Block RAM (RAMB36E1)	1	1
DSP slices	0	0

5.2 Timing and I/O Delay

All timing values are from the synthesis run described above: the per-port values are unconstrained worst-case boundary path delays, and the internal $F_{\max}$ is set by the register-to-register critical path. Under $\text{REG_IO} = 1$ every data port is at logic depth 0 except the state-management inputs and output_ready (one level), zeroize and resetn (two levels), and start and input_valid/input_final (three levels, the back-pressure decode); the mode input is directly registered (depth 0) in both configurations.

Table 7: Timing and I/O boundary characteristics on the Artix-7 target.

Characteristic	REG_IO = 1 (registered)	REG_IO = 0 (combinational)
Internal Fmax (reg-to-reg, SHA-2 round path)	132.8 MHz	132.7 MHz
Worst input port → first FF	~2.5 ns (2 levels, resetn)	~5.8 ns (deep byte_len count path)
Worst last FF → output port (clock-to-out)	~1.0 ns (0 levels)	~5.4 ns (output_data)
Combinational input → output path	none	present
I/O latency overhead (start → first output)	up to +3 cycles	none

5.3 ASIC Area (Gate Equivalents)

The area below is a technology-independent estimate produced by the open-source ASIC flow (make synth-yosys): yosys with the slang SystemVerilog frontend, mapped to the Nangate 45 nm Open Cell Library at the typical corner with module boundaries preserved. To keep the result fabric-neutral, the design is taken in its generic configuration: $\text{ADDER_IMPL} = \text{GENERIC}$ (no Xilinx CARRY4) and $\text{ROM_USE_BRAM} = 0$ (round constants as standard-cell logic). One gate equivalent (GE) corresponds to the area of a two-input NAND (NAND2_X1 , $0.798 \mu\text{m}^2$). The mapping is unconstrained (no clock or timing constraints), so these counts are not directly comparable with the Artix-7 LUT/FF values of Section 5.1.

Table 8: ASIC area estimate in gate equivalents (Nangate 45 nm, typical corner).

Component	REG_IO = 1 (kGE)	REG_IO = 0 (kGE)
Hash core — control + datapath (excl. SHA-2 adders)	45.7	43.2
SHA-2 round adders (13 dual-mode, 64-bit or 2×32 -bit)	6.3	6.3
Padding front-end (essec_sharmony_pad)	3.2	2.5
Round-constants ROM (distributed)	1.7	1.7
Top-level I/O glue	0.1	0.1
Total	57.0	53.8

5.4 Latency

The cycle counts below are measured in simulation for a single-block hash (SHA-224, one 512-bit block) and cross-checked against the performance test:

Table 9: Single-block latency (SHA-224, one 512-bit block).

Latency	REG_IO = 1	REG_IO = 0
Core active (FSM, <code>core_fsm_q</code> $\neq$ idle)	72 cycles	72 cycles
start $\rightarrow$ first output_valid	69 cycles	66 cycles
start $\rightarrow$ busy deasserted	74 cycles	73 cycles

The 72-cycle core-active latency equals the FSM latency reported by the performance test and does not depend on REG_IO, since the I/O boundary registering leaves the core’s cycle count unchanged. Selecting REG_IO = 1 adds three cycles to the host-visible start-to-first-output latency (one input and two output register stages). For longer messages the latency grows by the per-block cost; the performance test reports the per-mode setup, per-block, and throughput values. Note that the performance log’s end-to-end latency counts to the last busy-high cycle, while the table above counts to the deassertion edge of busy, one cycle later.

5.5 Throughput

Table 10 lists the steady-state throughput per mode. Throughput is defined as $TP = r \cdot f_{\text{clk}} / c_{\text{blk}}$, where r is the message-block (rate) size in bits and c_{blk} the steady-state cycle cost per block (the LAT column); the steady-state cost is measured as the cycle difference between a 64-block and a 32-block transaction divided by 32, so the fixed setup and drain overheads cancel. Rates are evaluated at the 132 MHz reference clock of the performance characterization. The TP/LUT column normalizes throughput to the 5873-LUT reference Artix-7 implementation (REG_IO = 1, Section 5.1). Verified by run-performance.

In the SHA-224/SHA-256 modes the core always runs both 32-bit lanes in lockstep, so a single-stream hash and a duet execute the same number of cycles; the duet rows simply account for both message streams, doubling the effective rate. The cSHAKE modes use the same rate and permutation as the corresponding SHAKE modes and share their rows.

Table 10: Throughput per mode at the 132 MHz reference clock; TP/LUT is normalized to the 5873-LUT reference implementation (REG_IO = 1).

Mode	LAT [CC]	f_{clk} [MHz]	TP [Mbps]	TP/LUT
SHA-224 / SHA-256 (single stream)	69	132	979	0.17
SHA-224 / SHA-256 (duet)	69	132	1959	0.33
SHA-384 / SHA-512 / SHA-512/224 / SHA-512/256	85	132	1590	0.27
SHA3-224	42	132	3621	0.62
SHA3-256	41	132	3503	0.60
SHA3-384	37	132	2968	0.51
SHA3-512	33	132	2304	0.39
SHAKE128 / cSHAKE128 (absorb)	45	132	3942	0.67
SHAKE256 / cSHAKE256 (absorb)	41	132	3503	0.60

For the XOF modes the table gives the absorb-phase rate. During the squeeze phase, output is streamed 64 bits per beat, and each additional r -bit output block requires one further KECCAK permutation.

6 Standards Compliance

Table 11 lists the standards applicable to SHARMONY and the conformance status of the design. Conformance is established by verification against the official test vectors (Section 7); no formal certification, such as CAVP or CMVP validation, is claimed. Regulatory directives for physical products do not apply to an RTL deliverable (Section 1).

Table 11: Applicable standards and conformance status.

Standard / Directive	Applicability	Status
FIPS 180-4 (SHS)	SHA-2 algorithm conformance	Verified against CAVP test vectors (Verilator simulation)
FIPS 202 (SHA-3)	SHA-3 / SHAKE algorithm conformance	Verified against CAVP test vectors (Verilator simulation)

7 Test and Verification Requirements

7.1 Verification Methodology

Verification is based on NIST CAVP test vectors: SHAVS vectors for SHA-2, and SHA3VS vectors for SHA-3 and SHAKE. cSHAKE128/256 are checked against the NIST SP 800-185 sample values together with a reference-derived empty-message vector. Tests are registered as C++ scenarios under `tb/src/tests/*_test.cpp` and run on a Verilator-based simulation flow: the RTL is compiled into a C++ model driven by a shared bus-functional driver (`SharmonyDriver`), and tests self-register through `TestRegistry`. The response vectors are stored under `tb/src/tests/data/*_rsp`.

Tooling requirements: Verilator 5.048 and GNU Make. Appendix A.3 gives the run commands and the locations of the generated artifacts.

7.2 Available Tests

Per-mode CAVP KAT scenarios exist for every supported variant (`shortmsg`, `longmsg`, and `monte` for each of SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, SHA-512/256, SHA3-224, SHA3-256, SHA3-384, SHA3-512, SHAKE128, and SHAKE256, plus `variableout` for the SHAKE modes and dedicated cSHAKE128/256 known-answer tests). In addition, the following cross-cutting scenarios are registered:

Table 12: Available tests in the verification suite.

Test	Purpose
<code>shortmsg</code>	Tests short message hashing and padding paths
<code>longmsg</code>	Tests long multi-block message processing
<code>interface</code>	Tests streaming and control interface operation
<code>error</code>	Verifies invalid input and error handling paths
<code>performance</code>	Measures throughput, latency, and cycle counts
<code>nightly</code>	Runs the complete regression test suite
<code>midstate</code>	Tests mid-state save/load and internal state caching
<code>zeroize</code>	Verifies zeroization behavior corner cases

All listed tests pass.

7.3 Verification Matrix

Table 13: Verification matrix: requirement coverage by test.

Req. ID	Verification Activity	Method	Procedure	Result
REQ-HW-001	SHA-2 CAVP KATs (shortmsg, longmsg, Monte Carlo) per mode	Test	run-sha*	Pass
REQ-HW-002	SHA-3/SHAKE CAVP KATs incl. variable output lengths	Test	run-sha3_*, run-shake*	Pass
REQ-HW-003	cSHAKE128/256 KATs (SP 800-185 samples + reference-derived)	Test	run-cshake128, run-cshake256	Pass
REQ-HW-004	Duet execution scenarios	Test	run-interface, run-nightly	Pass
REQ-HW-005	Stream handshake and back-pressure stress	Test	run-interface	Pass
REQ-HW-006	Mid-state save/load and on-chip caching (incl. duet)	Test	run-midstate	Pass
REQ-HW-007	Zeroization corner cases	Test	run-zeroize	Pass
REQ-HW-008	Protocol-violation and invalid-access scenarios	Test	run-error, run-sha3cache	Pass

Acknowledgement

This work has received funding from the European Union's Horizon Europe research and innovation programme under grant agreement No. 101225722 (FORTRESS). Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union. Neither the European Union nor the granting authority can be held responsible for them.

Copyright and Licensing

Copyright © 2025–2026 Universität der Bundeswehr München / Forschungsinstitut CODE, ESSEC Lab.

The SHARMONY RTL, its verification environment, and this specification are released under the Apache License, Version 2.0 (SPDX: Apache-2.0); a copy of the license is available at <https://www.apache.org/licenses/LICENSE-2.0>. Unless required by applicable law or agreed to in writing, the work is distributed on an “AS IS” basis, without warranties or conditions of any kind, either express or implied.

The bundled Xilinx carry-chain primitive simulation models (CARRY4.v, CARRY8.v, MUXCY.v) are third-party components: they are copyrighted by Xilinx, Inc. and licensed under the Apache License, Version 2.0, as stated in their file headers; the copyright statement above does not extend to them.

8 References

- [1] L. Anwar, C. A. Lara-Nino, J.-Y. Park, and M. Hutter, “SHARMONY: Composing SHA-2 and SHA-3 hardware for crypto-agile PQC,” To appear in IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES), 2026(4), 2026.
- [2] National Institute of Standards and Technology, “Secure hash standard (SHS),” U.S. Department of Commerce, Federal Information Processing Standards Publication FIPS PUB 180-4, 2015. DOI: 10.6028/NIST.FIPS.180-4.
- [3] National Institute of Standards and Technology, “SHA-3 standard: Permutation-based hash and extendable-output functions,” U.S. Department of Commerce, Federal Information Processing Standards Publication FIPS PUB 202, 2015. DOI: 10.6028/NIST.FIPS.202.
- [4] National Institute of Standards and Technology, “Stateless hash-based digital signature standard (SLH-DSA),” U.S. Department of Commerce, Federal Information Processing Standards Publication FIPS PUB 205, 2024. DOI: 10.6028/NIST.FIPS.205.
- [5] L. E. Bassham and T. A. Hall, “The secure hash algorithm validation system (SHAVS),” National Institute of Standards and Technology, Cryptographic Algorithm Validation Program, Tech. Rep., 2012.
- [6] L. E. Bassham and T. A. Hall, “The SHA-3 validation system (SHA3VS),” National Institute of Standards and Technology, Cryptographic Algorithm Validation Program, Tech. Rep., 2016.
- [7] A. Huelsing, D. Butin, S.-L. Gazdag, J. Rijneveld, and A. Mohaisen, “XMSS: Extended merkle signature scheme,” Internet Engineering Task Force (IETF), Request for Comments RFC 8391, 2018. DOI: 10.17487/RFC8391.
- [8] J. Kelsey, S.-j. Chang, and R. Perlner, “SHA-3 derived functions: cSHAKE, KMAC, TupleHash and ParallelHash,” National Institute of Standards and Technology, NIST Special Publication 800-185, 2016. DOI: 10.6028/NIST.SP.800-185.

A Appendices

A.1 Schematic References

The interface timing diagrams are collected in Section 4.4; the top-level block diagram is Figure 1. Figures 14 and 15 show the two halves of the shared 25×64 -bit register bank in detail; the registers R_i and H_i correspond to $R_q[i]$ and $H_q[i]$ in the RTL and in the waveforms of Sections 4.4 and 4.6. Figure 16 shows the memory map of the shared constants ROM.

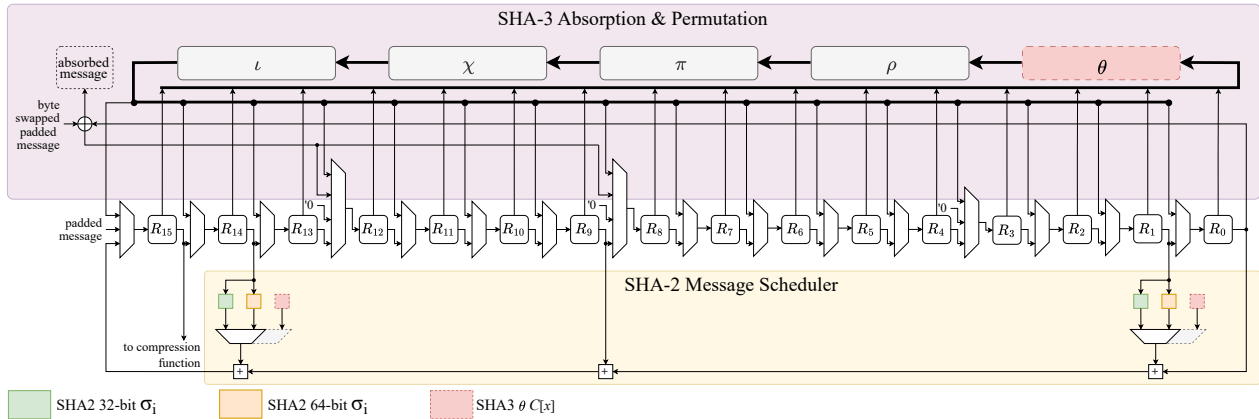


Figure 14: Message-scheduler half of the shared register bank (R_0 – R_{15}): the SHA-2 message schedule (32-bit and 64-bit σ_i paths) and the SHA-3 absorption path share the same registers, with the KECCAK permutation plane (θ , ρ , π , χ , l) overlaid.

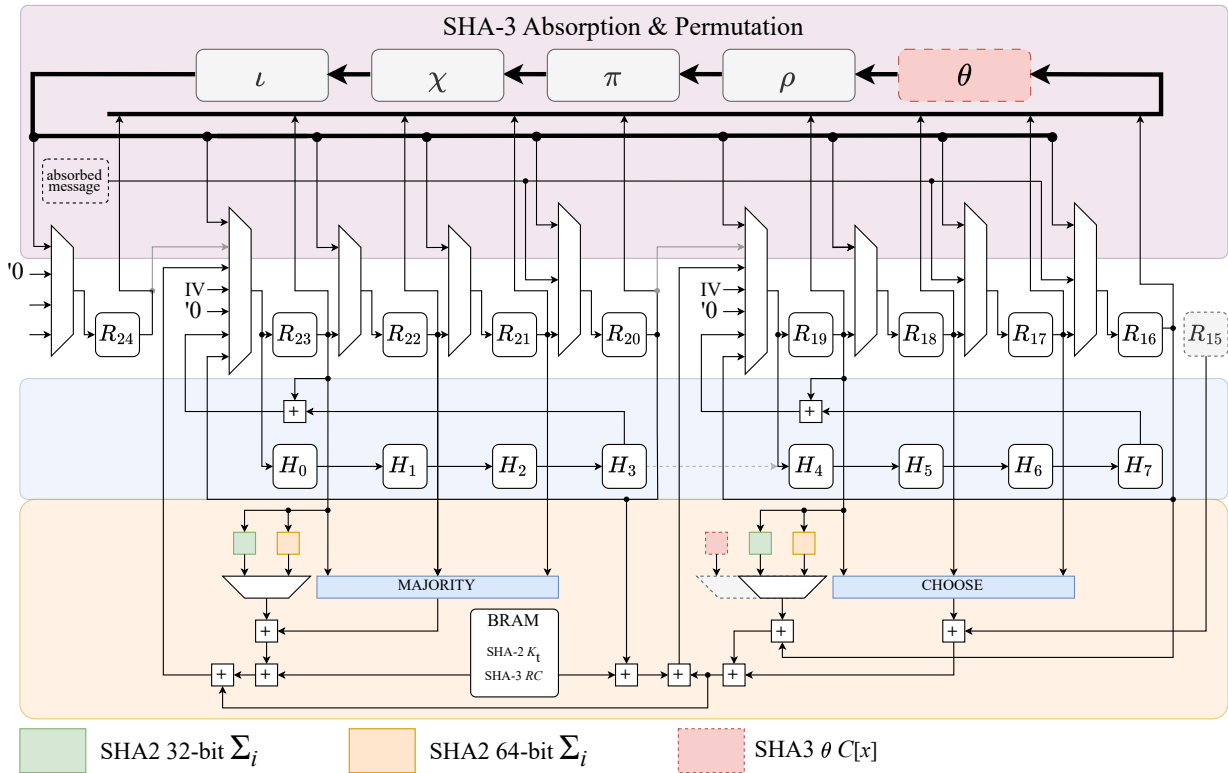


Figure 15: Compression half of the shared register bank (R_{16} – R_{24}) and the digest bank H_0 – H_7 : the SHA-2 working variables (a at R_{23} , e at R_{19}), the Σ_i , MAJORITY, and CHOOSE logic, the shared constants BRAM (SHA-2 K_t , SHA-3 RC), and the overlaid KECCAK permutation.

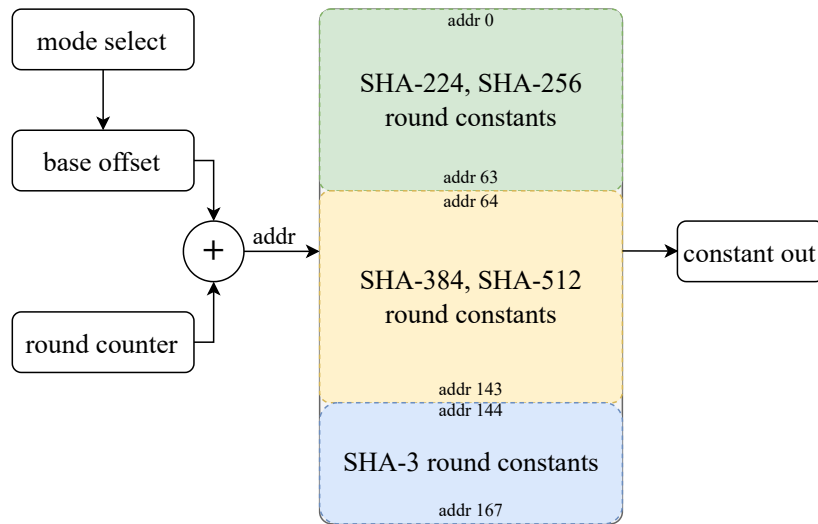


Figure 16: Memory map of the shared constants ROM: the mode-selected base offset plus the round counter address a single ROM holding the SHA-224/256 round constants (addresses 0–63), the SHA-384/512 round constants (64–143), and the SHA-3 round constants (144–167).

A.2 Repository Structure

```

SHARMONY/
├── docs/
│   ├── figures/           # block diagrams and figures
│   ├── waveforms/        # rendered timing-diagram figures (PDF)
│   └── sharmoney_hw_spec.pdf # hardware specification document
├── hw/
│   └── sharmoney/         # SHARMONY RTL source files
│       ├── essec_sharmoney_*.sv # pkg, top, core, pad, rom, adders
│       └── CARRY4.v, CARRY8.v, MUXCY.v # Xilinx carry-primitive sim models (Apache-2.0, see file headers)
├── synth/
│   ├── sharmoney_top/     # out-of-context synthesis flows (Vivado / yosys)
│   ├── lib/               # SHARMONY core OOC synth (make synth-vivado)
│   ├── work/              # Nangate45 liberty goes here (not bundled, see Requirements)
│   ├── ge_table.awk       # generated reports / checkpoints – gitignored
│   ├── ge_table.awk       # builds the ASIC gate-equivalent (kGE) area table
│   └── vivado_tables.awk  # formats Vivado resource/timing/I-O tables
├── tb/
│   ├── src/               # active Verilator/C++ simulation flow
│   │   ├── sharmoney_verilator_wrapper.sv
│   │   ├── main.cpp
│   │   ├── SharmonyDriver.{cpp,hpp}
│   │   ├── TestRegistry.{cpp,hpp}
│   │   └── RspParser.{cpp,hpp}
│   ├── tests/
│   │   ├── common/        # shared C++ test utilities
│   │   ├── data/*.rsp     # NIST CAVP response vectors
│   │   └── *_test.cpp      # registered test scenarios
│   └── work/              # generated build, log, VCD, and report output – gitignored
├── LICENSE                # Apache License, Version 2.0
├── Makefile               # build, lint, test, waveform, and clean targets
└── README.md

```

A.3 Running the Test Suite

Run commands from the project root (the directory containing the Makefile):

```

make list-tests           # list available tests
make run-sha256_shortmsg  # quick run
make run-sha512_shortmsg VCD=1 # run with VCD waveform
make run-nightly          # full regression suite

```

Generated artifacts: log under `tb/work/logs/<TESTNAME>/log` and waveform under `tb/work/vcd/`.